

The **LIKE operator in MySQL** is used for **pattern matching** in **SELECT, UPDATE, or DELETE** queries. It allows you to search for a specified pattern in a column.

◆ Syntax

```
SELECT column_name
FROM table_name
WHERE column_name LIKE pattern;
```

◆ Wildcards Used with LIKE

Wildcard	Description
%	Represents zero or more characters
_	Represents exactly one character

◆ Examples

1. Names starting with 'A'

```
SELECT * FROM Student
WHERE name LIKE 'A%';
```

👉 Matches: Amit, Anu, Arjun

2. Names ending with 'a'

```
SELECT * FROM Student
WHERE name LIKE '%a';
```

👉 Matches: Rita, Sita

3. Names containing 'an'

```
SELECT * FROM Student
WHERE name LIKE '%an%';
```

👉 Matches: Anand, Manoj

4. Names with exactly 4 characters

```
SELECT * FROM Student
WHERE name LIKE '____';
```

👉 Matches: John, Ravi

5. Second letter is 'a'

```
SELECT * FROM Student
WHERE name LIKE '_a%';
```

👉 Matches: Raj, Karan

◆ Case Sensitivity

- In MySQL, `LIKE` is **case-insensitive** by default (depends on collation).
- To make it case-sensitive, use:

```
SELECT * FROM Student
WHERE name LIKE BINARY 'A%';
```

◆ Using NOT LIKE

```
SELECT * FROM Student
WHERE name NOT LIKE 'A%';
```

👉 Returns names that **do NOT** start with 'A'

◆ Real-time Example

```
SELECT * FROM Employee
WHERE department LIKE '%Tech%';
```

👉 Finds departments like Technology, Tech Support

◆ Key Points

- `%` → any number of characters

- `_` → exactly one character
 - Used mainly in `WHERE` clause
 - Works with `SELECT`, `UPDATE`, `DELETE`
 - Aggregate functions in MySQL are used to **perform calculations on multiple rows and return a single value**. They are heavily used in reporting, analytics, and real-time data summaries.
-

Updating **multiple columns using `CASE` in MySQL** is a very useful technique when you want to apply **different logic to different rows in a single query**.

I'll explain it step-by-step in simple English.

◆ What is `CASE` in `UPDATE`?

👉 `CASE` works like **IF–ELSE** logic inside SQL.

“If condition is true → do this, otherwise → do something else”

◆ Basic Syntax

```
UPDATE table_name
SET
    column1 = CASE
        WHEN condition1 THEN value1
        WHEN condition2 THEN value2
        ELSE default_value
    END,
    column2 = CASE
        WHEN condition1 THEN value1
        WHEN condition2 THEN value2
        ELSE default_value
    END;

```

◆ Example Table: `Employee`

emp_id name department salary

1	Anu	HR	20000
2	Ravi	IT	30000
3	Kiran	IT	25000
4	Meena	HR	22000
5	Arjun	Sales	28000

◆ Example 1: Update Salary and Department Together

👉 Requirement:

- If department = IT → increase salary to 35000 and set department = "Tech"
- If department = HR → salary = 25000 and department = "Admin"

```
UPDATE Employee
SET
    salary = CASE
        WHEN department = 'IT' THEN 35000
        WHEN department = 'HR' THEN 25000
        ELSE salary
    END,
    department = CASE
        WHEN department = 'IT' THEN 'Tech'
        WHEN department = 'HR' THEN 'Admin'
        ELSE department
    END;

```

◆ Example 2: Update Based on emp_id

```
UPDATE Employee
SET
    salary = CASE
        WHEN emp_id = 1 THEN 21000
        WHEN emp_id = 2 THEN 32000
        ELSE salary
    END,
    department = CASE
        WHEN emp_id = 1 THEN 'Finance'
        WHEN emp_id = 2 THEN 'Support'
        ELSE department
    END;

```

◆ Example 3: Use WHERE with CASE (Best Practice)

```
UPDATE Employee
SET
    salary = CASE
        WHEN department = 'IT' THEN salary + 5000
        WHEN department = 'Sales' THEN salary + 3000
        ELSE salary
    END,
    department = CASE
        WHEN department = 'Sales' THEN 'Marketing'
        ELSE department
    END
WHERE department IN ('IT', 'Sales');
```

-

- Common Aggregate Functions

Function	Description
----------	-------------

COUNT ()	Counts number of rows
----------	-----------------------

SUM ()	Calculates total
--------	------------------

AVG ()	Calculates average
--------	--------------------

MIN ()	Finds minimum value
--------	---------------------

MAX ()	Finds maximum value
--------	---------------------

-

- Practical Example Table: `Employee`

emp_id	name	department	salary
--------	------	------------	--------

1	Anu	HR	20000
---	-----	----	-------

2	Ravi	IT	30000
---	------	----	-------

3	Kiran	IT	25000
---	-------	----	-------

4	Meena	HR	22000
---	-------	----	-------

5	Arjun	Sales	28000
---	-------	-------	-------

-

- 1. COUNT() – Total Employees

- ```
SELECT COUNT(*) AS total_employees
FROM Employee;
```

-  Output: 5

- 

-  2. SUM() – Total Salary

- ```
SELECT SUM(salary) AS total_salary
FROM Employee;
```

-  Output: 125000

-

- 3. AVG() – Average Salary

- ```
SELECT AVG(salary) AS avg_salary
FROM Employee;
```

-  Output: 25000

-

-  4. MIN() – Lowest Salary
  - ```
SELECT MIN(salary) AS lowest_salary  
FROM Employee;
```
 -  Output: 20000
-

-  5. MAX() – Highest Salary
- ```
SELECT MAX(salary) AS highest_salary
FROM Employee;
```
-  Output: 30000

Let's break down **GROUP BY** and **HAVING** in *simple, normal English* with clear examples.

---

## What is `GROUP BY`?

 **`GROUP BY` is used to group rows that have the same values in a column.**

Think of it like:

“Put similar data into one group, then calculate something for each group.”

---

## Example Table: `Employee`

**emp\_id name department salary**

|   |       |       |       |
|---|-------|-------|-------|
| 1 | Anu   | HR    | 20000 |
| 2 | Ravi  | IT    | 30000 |
| 3 | Kiran | IT    | 25000 |
| 4 | Meena | HR    | 22000 |
| 5 | Arjun | Sales | 28000 |

---

## Example 1: Group employees by department

```
SELECT department
FROM Employee
GROUP BY department;
```

 Output:

- HR
- IT
- Sales

👉 It just shows **unique groups**

---

## ◆ Example 2: Total salary per department

```
SELECT department, SUM(salary)
FROM Employee
GROUP BY department;
```

👉 Output:

**department total\_salary**

|       |       |
|-------|-------|
| HR    | 42000 |
| IT    | 55000 |
| Sales | 28000 |

👉 Here:

- All HR salaries are grouped → summed
  - All IT salaries are grouped → summed
- 

## ◆ Example 3: Count employees in each department

```
SELECT department, COUNT(*)
FROM Employee
GROUP BY department;
```

---

### ◆ What is `HAVING`?

👉 **HAVING is used to filter grouped data (after GROUP BY).**

Think of it like:

“After grouping, now filter the groups based on some condition.”

---

## ◆ Example: Show only departments with total salary > 50,000

```
SELECT department, SUM(salary) AS total_salary
FROM Employee
GROUP BY department
HAVING SUM(salary) > 50000;
```

👉 Output:

- IT (55000)

👉 Explanation:

- First groups are created
- Then condition is applied on those groups

---

## ◆ Difference Between `WHERE` and `HAVING`

| Feature                        | WHERE           | HAVING           |
|--------------------------------|-----------------|------------------|
| Used for                       | Filtering rows  | Filtering groups |
| Applied                        | Before grouping | After grouping   |
| Works with aggregate functions | ✗ No            | ✓ Yes            |

---

## ◆ Example to Understand Difference

✗ Wrong:

```
SELECT department, SUM(salary)
FROM Employee
WHERE SUM(salary) > 50000
GROUP BY department;
```

👉 Error because `WHERE` cannot use aggregate functions

---

✓ Correct:

```
SELECT department, SUM(salary)
FROM Employee
GROUP BY department
HAVING SUM(salary) > 50000;
```



---

## ◆ Real-Life Analogy

Imagine a classroom:

- `GROUP BY` → Group students by **section**
- `SUM/COUNT` → Count students in each section
- `HAVING` → Show only sections with **more than 30 students**

---

## ◆ Combined Example (Very Important)

```
SELECT department, AVG(salary) AS avg_salary
FROM Employee
WHERE salary > 20000
GROUP BY department
HAVING AVG(salary) > 25000;
```

👉 Step by step:

1. `WHERE` → remove employees with salary  $\leq 20000$
2. `GROUP BY` → group by department
3. `HAVING` → show only high average salary groups

---

## ◆ Key Points to Remember

- ✓ `GROUP BY` → makes groups
- ✓ `HAVING` → filters groups
- ✓ `WHERE` → filters rows
- ✓ Use aggregate functions with `GROUP BY`
- ✓ Use `HAVING` when condition involves `SUM`, `AVG`, etc.

Let's understand **ORDER BY in MySQL** in simple, clear English.

---

## ◆ What is `ORDER BY`?

👉 **`ORDER BY` is used to sort the result of a query.**

Think of it like:

“Arrange the data in a specific order (ascending or descending).”

---

## ◆ Syntax

```
SELECT column1, column2
FROM table_name
ORDER BY column_name [ASC | DESC];
```

---

## ◆ Types of Sorting

| Keyword | Meaning                                           |
|---------|---------------------------------------------------|
| ASC     | Ascending (small → big, A → Z) ( <i>default</i> ) |
| DESC    | Descending (big → small, Z → A)                   |

---

## ◆ Example Table: `Employee`

| emp_id | name  | department | salary |
|--------|-------|------------|--------|
| 1      | Anu   | HR         | 20000  |
| 2      | Ravi  | IT         | 30000  |
| 3      | Kiran | IT         | 25000  |
| 4      | Meena | HR         | 22000  |
| 5      | Arjun | Sales      | 28000  |

### ◆ 1. Sort by Salary (Ascending)

```
SELECT * FROM Employee
ORDER BY salary ASC;
```

👉 Output: 20000 → 22000 → 25000 → 28000 → 30000

---

### ◆ 2. Sort by Salary (Descending)

```
SELECT * FROM Employee
ORDER BY salary DESC;
```

👉 Output: 30000 → 28000 → 25000 → 22000 → 20000

---

### ◆ 3. Sort by Name (Alphabetical)

```
SELECT * FROM Employee
ORDER BY name;
```

👉 A → Z order

---

### ◆ 4. Sort by Multiple Columns

```
SELECT * FROM Employee
ORDER BY department ASC, salary DESC;
```

👉 Meaning:

- First sort by department
  - Then within each department, sort by salary (highest first)
- 

### ◆ 5. Using ORDER BY with GROUP BY

```
SELECT department, SUM(salary) AS total_salary
FROM Employee
GROUP BY department
ORDER BY total_salary DESC;
```

👉 Shows departments with highest total salary first

---

### ◆ 6. Using Column Position (Not Recommended but Possible)

```
SELECT name, salary FROM Employee
ORDER BY 2 DESC;
```

👉 2 means second column (salary)

---

### ◆ 7. Using ORDER BY with LIMIT (Very Important)

👉 Top N results

```
SELECT * FROM Employee
ORDER BY salary DESC
LIMIT 3;
```

👉 Top 3 highest-paid employees

---

## ◆ Execution Order (Important Concept)

FROM → WHERE → GROUP BY → HAVING → ORDER BY → LIMIT

👉 ORDER BY happens **after everything else**

## ◆ What is JOIN?

👉 A **JOIN** is used to **combine data from two or more tables** based on a related column.

Think of it like:

“Take data from different tables and connect them using a common field.”

---

## ◆ Example Tables

### 📄 Student

**student\_id name department**

|   |       |    |
|---|-------|----|
| 1 | Anu   | CS |
| 2 | Ravi  | IT |
| 3 | Kiran | CS |

---

### 📄 Course

**course\_id course\_name**

|     |      |
|-----|------|
| 101 | DBMS |
| 102 | Java |

---

### 📄 Enrollment

**enroll\_id student\_id course\_id**

|   |   |     |
|---|---|-----|
| 1 | 1 | 101 |
| 2 | 2 | 102 |

enroll\_id student\_id course\_id

3            3            101

---

## ◆ Types of JOIN

---

### ◆ 1. INNER JOIN

👉 Returns **only matching records** from both tables

```
SELECT Student.name, Course.course_name
FROM Student
INNER JOIN Enrollment
ON Student.student_id = Enrollment.student_id
INNER JOIN Course
ON Enrollment.course_id = Course.course_id;
```

👉 Output:

- Anu → DBMS
- Ravi → Java
- Kiran → DBMS

✓ Only matched data is shown

---

### ◆ 2. LEFT JOIN

👉 Returns:

- All records from **left table**
- Matching records from right table
- If no match → NULL

```
SELECT Student.name, Course.course_name
FROM Student
LEFT JOIN Enrollment
ON Student.student_id = Enrollment.student_id
LEFT JOIN Course
ON Enrollment.course_id = Course.course_id;
```

✓ Shows all students (even if not enrolled)

---

### ◆ 3. RIGHT JOIN

👉 Returns:

- All records from **right table**
- Matching from left table

```
SELECT Student.name, Course.course_name
FROM Student
RIGHT JOIN Enrollment
ON Student.student_id = Enrollment.student_id;
```

---

### ◆ 4. FULL JOIN (Concept)

👉 MySQL does **not support FULL JOIN directly**

But we can simulate:

```
SELECT * FROM Student
LEFT JOIN Enrollment ON Student.student_id = Enrollment.student_id

UNION

SELECT * FROM Student
RIGHT JOIN Enrollment ON Student.student_id = Enrollment.student_id;
```

✓ Combines LEFT + RIGHT JOIN

---

### ◆ 5. CROSS JOIN

👉 Returns **all combinations**

```
SELECT Student.name, Course.course_name
FROM Student
CROSS JOIN Course;
```

👉 Output:

- Anu → DBMS
- Anu → Java
- Ravi → DBMS
- Ravi → Java
- ...

✓ Every student with every course

---

## 6. SELF JOIN

👉 Join a table with itself

### Example Table: Employee

**emp\_id name manager\_id**

1      A      NULL

2      B      1

3      C      1

```
SELECT e.name AS Employee, m.name AS Manager
FROM Employee e
LEFT JOIN Employee m
ON e.manager_id = m.emp_id;
```

👉 Output:

- B → A
- C → A